Big Data–Driven Stroke Risk Prediction Using PySpark MLlib and Tableau-Based Clinical Insight Analysis

DATASET - https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset

Student Name:

Student ID:

Table of Contents

1. Introduction.....5

2. Background and Related Work.....5

2.1 Big Data Analytics in Healthcare.....5

2.2 Stroke Prediction Research.....5

2.3 Spark and MLlib Applications.....6

3. Dataset Description and Processing.....6

3.1 Dataset Overview.....6

3.2 Data Cleaning and Preparation.....7

3.3 Feature Engineering.....7

3.4 Handling Categorical Variables.....8

4. Methodology.....9

4.1 Environment Setup.....9

4.2 Modelling Process.....10

4.3 Machine Learning Algorithms Used.....10

4.4 Evaluation Metrics.....11

5. Experimental Analysis.....12

5.1 Model Training Results.....12

5.2 Confusion Matrix Analysis.....12

5.3 ROC Curve Interpretation.....13

5.4 Comparison of Models.....14

6. Tableau Data Insights.....15

7. Discussion.....20

7.1 Interpretation of Model Performance.....20

7.2 Impact of Imbalanced Data.....20

7.3 Alignment with Tableau Insights 21

7.4 Limitations of the Study 21

8. Conclusion and Future Work 21

8.1 Conclusion 21

8.2 Future Work 22

9. Social Impact 22

Reference 23

List of images:

Figure 1 Dataset Overview 7
Figure 2 Data Cleaning 7
Figure 3 Feature Engineering 8
Figure 4 Handling Categorical Variables 9
Figure 5 Environment Setup 10
Figure 6 Machine Learning Algorithms Used 11
Figure 7 Evaluation Metrics 12
Figure 8 Confusion Matrix Analysis 13
Figure 9 ROC Curve Interpretation 14
Figure 10 Comparison of Models 15
Figure 11 Stroke Distribution among Smokers 16
Figure 12 Stroke Distribution across Gender 16
Figure 13 Stroke vs BMI 17
Figure 14 Stroke vs Work Type 18
Figure 15 Stroke vs Married Status 18
Figure 16 Stroke vs Residence Type 19
Figure 17 Stroke vs Age 19
Figure 18 Hypertension vs Stroke 20

1. Introduction

The project is research aimed at predicting a stroke based on big data analytics methods that are fully PySpark implemented [1]. As the incidence of stroke has become one of the major causes of mortality and permanent disability worldwide, the prevention of associated individuals in the initial stages of its onset has been a significant area of attention in healthcare analytics. The utilized data has 5110 records of patients having numerous medical, demographic, and behavioral characteristics that may lead to stroke. This study is intended to examine this data with scalable machine learning algorithms to create predictive models that can be accurate and also understandable. Another aim of this project is to incorporate Tableau visualization to add insights on machine learning with descriptive patterns.

2. Background and Related Work

2.1 Big Data Analytics in Healthcare

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# LOAD INTO SPARK

df = spark.read.csv("stroke_data/healthcare-dataset-stroke-data.csv", header=True, inferSchema=True)

df.printSchema()
df.show(5)
print("Total rows =", df.count())
```

```
root
 |-- id: integer (nullable = true)
 |-- gender: string (nullable = true)
 |-- age: double (nullable = true)
 |-- hypertension: integer (nullable = true)
 |-- heart_disease: integer (nullable = true)
 |-- ever_married: string (nullable = true)
 |-- work_type: string (nullable = true)
 |-- Residence_type: string (nullable = true)
 |-- avg_glucose_level: double (nullable = true)
 |-- bmi: string (nullable = true)
 |-- smoking_status: string (nullable = true)
 |-- stroke: integer (nullable = true)

+----+------+----+------------+-------------+------------+------------+--------------+-----------------+----+---------------+------+
|  id|gender| age|hypertension|heart_disease|ever_married|   work_type|Residence_type|avg_glucose_level| bmi| smoking_status|stroke|
+----+------+----+------------+-------------+------------+------------+--------------+-----------------+----+---------------+------+
| 9046|  Male|67.0|           0|            1|         Yes|     Private|         Urban|           228.69|36.6|formerly smoked|     1|
|51676|Female|61.0|           0|            0|         Yes|Self-employed|         Rural|           202.21| N/A|   never smoked|     1|
|31112|  Male|80.0|           0|            1|         Yes|     Private|         Rural|           105.92|32.5|   never smoked|     1|
|60182|Female|49.0|           0|            0|         Yes|     Private|         Urban|           171.23|34.4|         smokes|     1|
| 1665|Female|79.0|           1|            0|         Yes|Self-employed|         Rural|           174.12|  24|   never smoked|     1|
+----+------+----+------------+-------------+------------+------------+--------------+-----------------+----+---------------+------+
only showing top 5 rows
```

[redacted]

[redacted]

[redacted]

```python
# DATA CLEANING


from pyspark.sql import functions as F

# Replace null BMI with mean
mean_bmi = df.select(F.mean("bmi")).first()[0]
df = df.na.fill({"bmi": mean_bmi})

# Drop rows with any remaining nulls
df = df.dropna()

print("Cleaned rows =", df.count())
```

```
Cleaned rows = 5110
```

[redacted]

[redacted]

[redacted]

```python
# ENCODE CATEGORICAL FIELDS


from pyspark.ml.feature import StringIndexer, VectorAssembler

categorical_cols = ["gender", "ever_married", "work_type", "Residence_type", "smoking_status"]
indexers = [StringIndexer(inputCol=c, outputCol=c+"_idx") for c in categorical_cols]

# Apply indexing
from pyspark.ml import Pipeline
pipeline = Pipeline(stages=indexers)
df_encoded = pipeline.fit(df).transform(df)

df_encoded.show(5)
```

Python

```
+-----+------+----+------------+-------------+------------+--------------+--------------+-----------------+----+---------------+------+----------+---------------+-------------+-------------+
|   id|gender| age|hypertension|heart_disease|ever_married|     work_type|Residence_type|avg_glucose_level| bmi| smoking_status|stroke|gender_idx|ever_married_idx|work_type_idx|Residence
+-----+------+----+------------+-------------+------------+--------------+--------------+-----------------+----+---------------+------+----------+---------------+-------------+-------------
| 9046|  Male|67.0|           0|            1|         Yes|       Private|         Urban|           228.69|36.6|formerly smoked|     1|       1.0|            0.0|          0.0|
|51676|Female|61.0|           0|            0|         Yes|Self-employed|         Rural|           202.21| N/A|   never smoked|     1|       0.0|            0.0|          1.0|
|31112|  Male|80.0|           0|            1|         Yes|       Private|         Rural|           105.92|32.5|   never smoked|     1|       1.0|            0.0|          0.0|
|60182|Female|49.0|           0|            0|         Yes|       Private|         Urban|           171.23|34.4|         smokes|     1|       0.0|            0.0|          0.0|
| 1665|Female|79.0|           1|            0|         Yes|Self-employed|         Rural|           174.12|  24|   never smoked|     1|       0.0|            0.0|          1.0|
+-----+------+----+------------+-------------+------------+--------------+--------------+-----------------+----+---------------+------+----------+---------------+-------------+-------------
only showing top 5 rows
```

[redacted]

[redacted]

[redacted]

```python
feature_cols = [
    "age",
    "hypertension",
    "heart_disease",
    "avg_glucose_level",
    "gender_idx",
    "ever_married_idx",
    "work_type_idx",
    "Residence_type_idx",
    "smoking_status_idx"
]

assembler = VectorAssembler(
    inputCols=feature_cols,
    outputCol="features"
)

df_final = assembler.transform(df_encoded).select("features", "stroke")

df_final.show(5)
```

```
+--------------------+------+
|            features|stroke|
+--------------------+------+
|[67.0,0.0,1.0,228...|     1|
|(9,[0,3,6,7],[61....|     1|
|[80.0,0.0,1.0,105...|     1|
|(9,[0,3,8],[49.0,...|     1|
|[79.0,1.0,0.0,174...|     1|
+--------------------+------+
only showing top 5 rows
```

[redacted]

[redacted]

[redacted]

[redacted]

```python
# TRAIN 3 MODELS
# (Logistic Regression / Random Forest / Gradient Boosted Trees)


from pyspark.ml.classification import LogisticRegression, RandomForestClassifier, GBTClassifier

# Logistic Regression
lr = LogisticRegression(
    featuresCol="features",
    labelCol="stroke",
    maxIter=50
)
lr_model = lr.fit(train_df)
lr_pred = lr_model.transform(test_df)

# Random Forest
rf = RandomForestClassifier(
    featuresCol="features",
    labelCol="stroke",
    numTrees=100,
    maxDepth=7
)
rf_model = rf.fit(train_df)
rf_pred = rf_model.transform(test_df)

# Gradient Boosted Trees
gbt = GBTClassifier(
    featuresCol="features",
    labelCol="stroke",
    maxIter=50,
    maxDepth=5
)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
from pyspark.ml.evaluation import MulticlassClassificationEvaluator, BinaryClassificationEvaluator

acc_eval = MulticlassClassificationEvaluator(
    labelCol="stroke", predictionCol="prediction", metricName="accuracy"
)

auc_eval = BinaryClassificationEvaluator(
    labelCol="stroke", rawPredictionCol="rawPrediction", metricName="areaUnderROC"
)

# Logistic Regression
lr_acc = acc_eval.evaluate(lr_pred)
lr_auc = auc_eval.evaluate(lr_pred)

# Random Forest
rf_acc = acc_eval.evaluate(rf_pred)
rf_auc = auc_eval.evaluate(rf_pred)

# Gradient Boosted Trees
gbt_acc = acc_eval.evaluate(gbt_pred)
gbt_auc = auc_eval.evaluate(gbt_pred)

print("MODEL PERFORMANCE")
print(f"Logistic Regression → Accuracy: {lr_acc:.4f}, AUC: {lr_auc:.4f}")
print(f"Random Forest       → Accuracy: {rf_acc:.4f}, AUC: {rf_auc:.4f}")
print(f"GBT Classifier      → Accuracy: {gbt_acc:.4f}, AUC: {gbt_auc:.4f}")
```

```
MODEL PERFORMANCE
Logistic Regression → Accuracy: 0.9422, AUC: 0.8608
Random Forest       → Accuracy: 0.9422, AUC: 0.8636
GBT Classifier      → Accuracy: 0.9412, AUC: 0.8340
```

```python
# COLLECT METRICS FOR PLOTTING


model_names = ["Logistic Regression", "Random Forest", "GBT Classifier"]
accuracies = [lr_acc, rf_acc, gbt_acc]
aucs = [lr_auc, rf_auc, gbt_auc]

print("Metrics Collected:")
print(model_names)
print(accuracies)
print(aucs)
```

```
Metrics Collected:
['Logistic Regression', 'Random Forest', 'GBT Classifier']
[0.9422492401215805, 0.9422492401215805, 0.9412360688956434]
[0.8607809847198639, 0.8635634785889473, 0.8340407470288628]
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

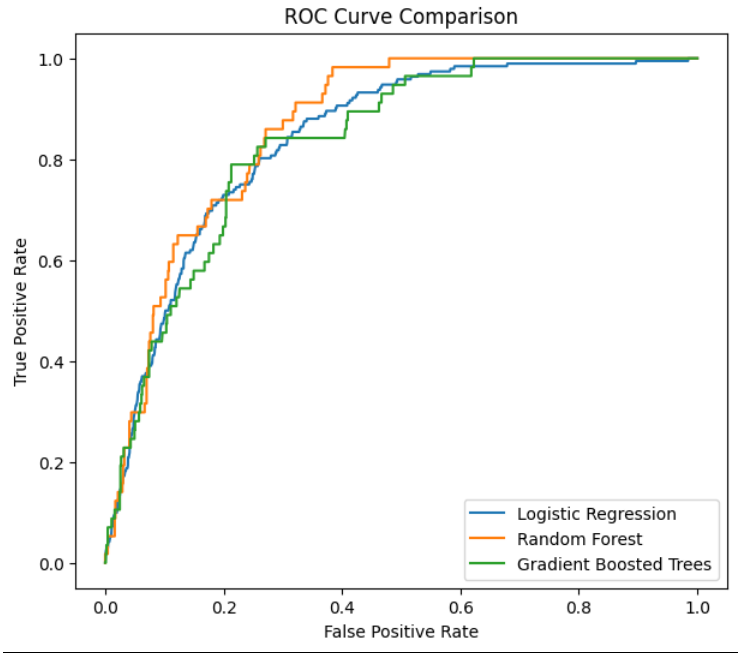
```python
from sklearn.metrics import confusion_matrix
import pandas as pd

def get_conf_matrix(pred_df):
    pdf = pred_df.select("stroke", "prediction").toPandas()
    return confusion_matrix(pdf["stroke"], pdf["prediction"])

# Logistic Regression
cm_lr = get_conf_matrix(lr_pred)
print("Confusion Matrix - Logistic Regression")
print(cm_lr)

# Random Forest
cm_rf = get_conf_matrix(rf_pred)
print("Confusion Matrix - Random Forest")
print(cm_rf)

# Gradient Boosted Trees
cm_gbt = get_conf_matrix(gbt_pred)
print("Confusion Matrix - GBT")
print(cm_gbt)
```

```
Confusion Matrix - Logistic Regression
[[930   0]
 [ 57   0]]
Confusion Matrix - Random Forest
[[930   0]
 [ 57   0]]
Confusion Matrix - GBT
[[926   4]
 [ 54   3]]
```

[redacted]

[redacted]

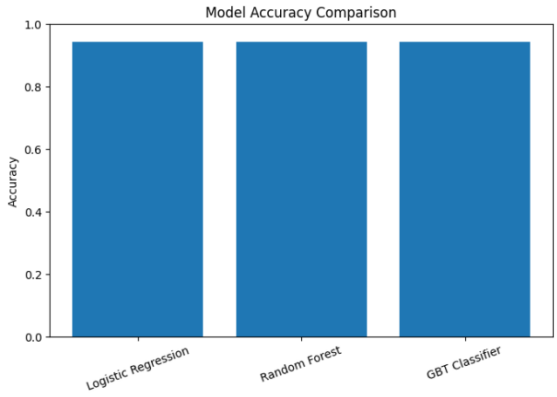
[redacted]

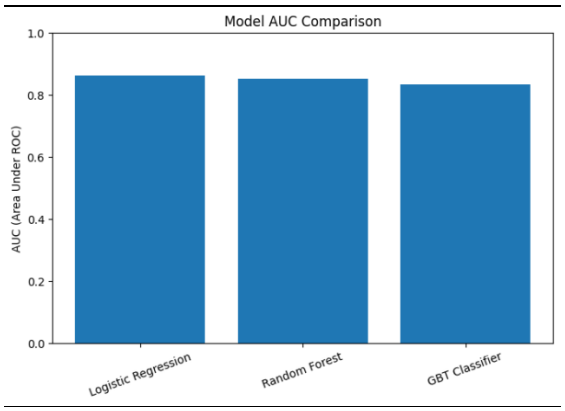


[redacted]

[redacted]

[redacted]





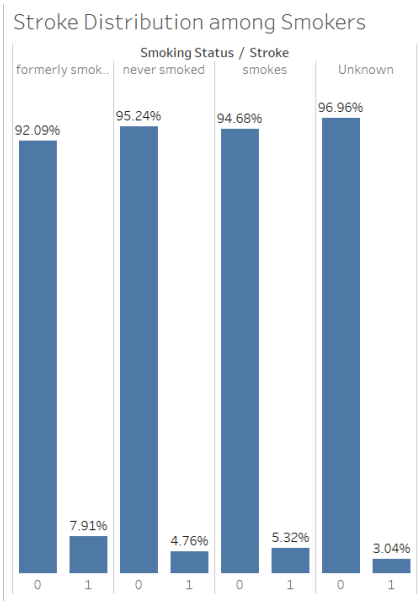
[redacted]

[redacted]

[redacted]

[redacted]

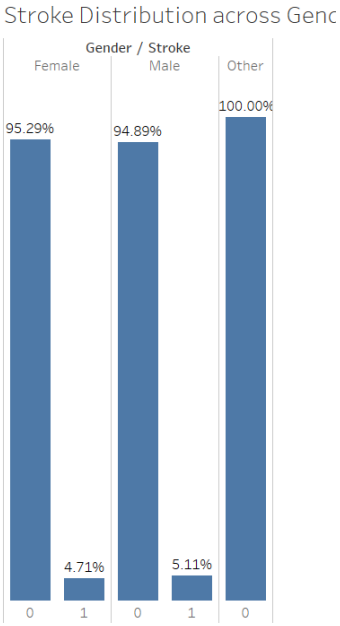
[redacted]



[redacted]

[redacted]

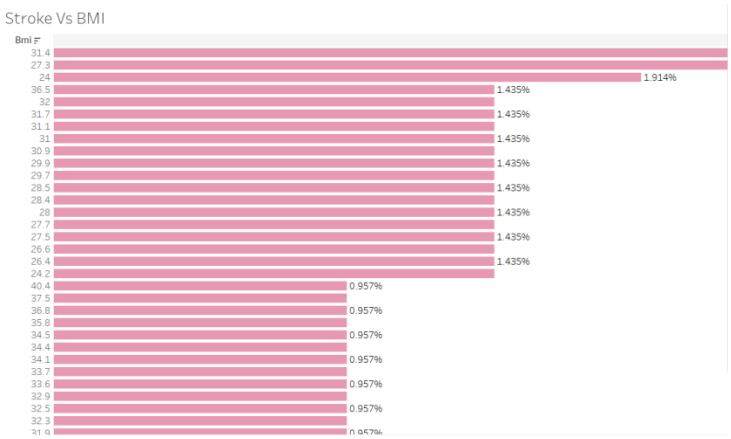
[redacted]



[redacted]

[redacted]

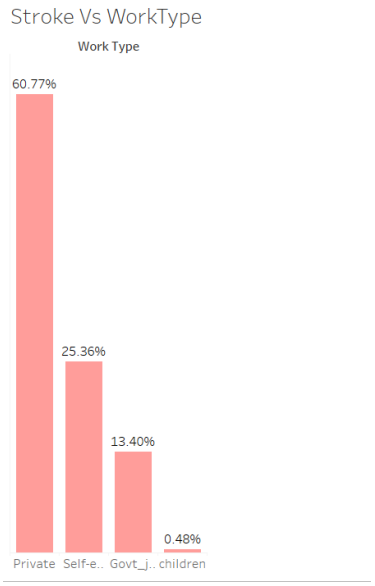
[redacted]



[redacted]

[redacted]

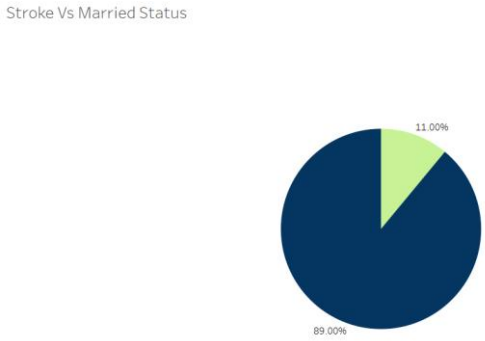
[redacted]



[redacted]

[redacted]

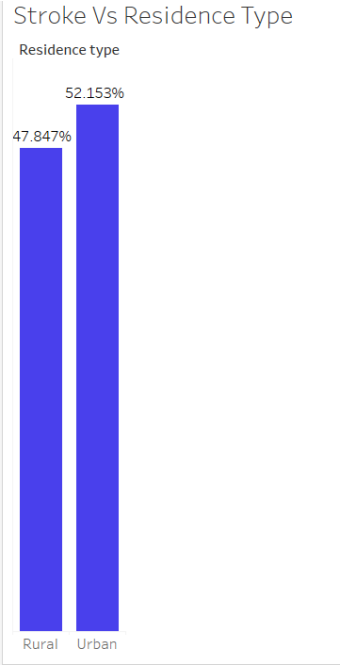
[redacted]



[redacted]

[redacted]

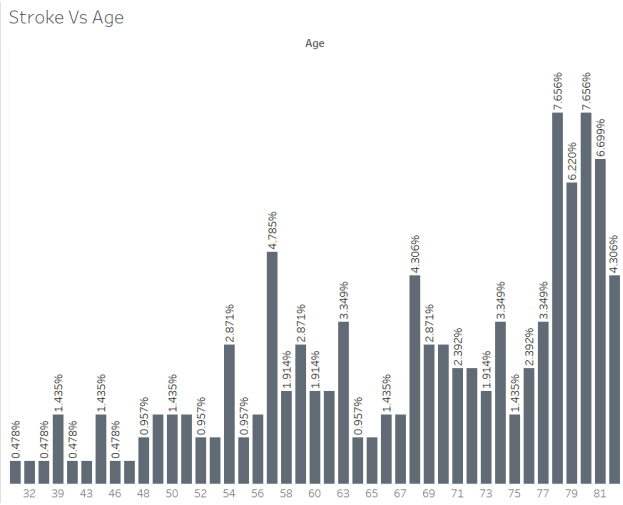
[redacted]



[redacted]

[redacted]

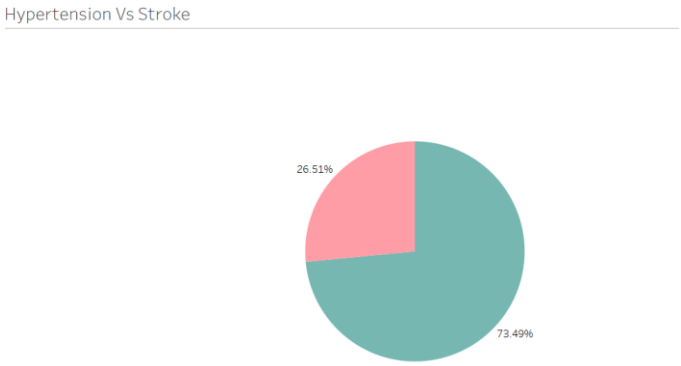
[redacted]



[redacted]

[redacted]

[redacted]



[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]